

OUTBOX PATTERN — DEMONSTRAÇÃO

Garantindo que nenhuma mensagem se perca

Como o sistema evita perder ou duplicar um evento de negócio, mesmo quando algo falha no meio do caminho.

Apresentação para liderança · agosto de 2026

O PROBLEMA

Duas ações, um único momento — e se uma falhar?

Toda vez que um sistema salva algo importante no banco de dados e precisa avisar outros sistemas sobre isso (um "evento"), existe um risco: as duas ações podem não acontecer juntas.

✗ Cenário 1

O dado é salvo no banco, mas o aviso nunca chega a ser enviado (o sistema caiu, a rede falhou). Outros sistemas nunca ficam sabendo — **informação perdida**.

✗ Cenário 2

O aviso é enviado, mas o dado não chega a ser salvo (a transação falhou depois). Outros sistemas reagem a algo que **nunca aconteceu de verdade**.

A SOLUÇÃO

Outbox Pattern: o aviso nasce junto com o dado

Em vez de salvar o dado e enviar o aviso como duas ações separadas, o sistema salva **os dois juntos, na mesma operação** — ou os dois acontecem, ou nenhum acontece. O envio de fato para os outros sistemas é feito logo depois, com garantia de que vai acontecer eventualmente, mesmo se algo falhar no meio do caminho.

✓ Sem perda

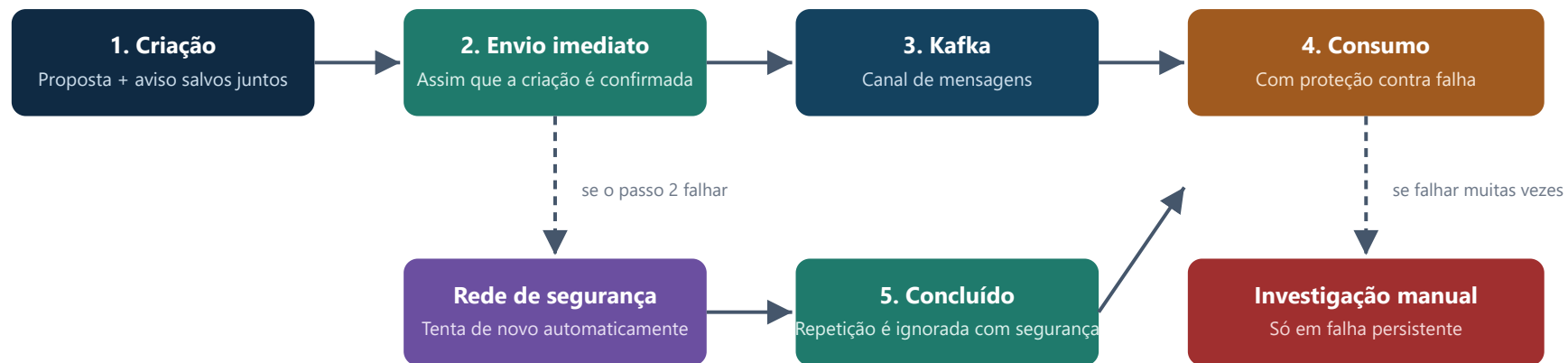
O "aviso pendente" fica registrado no banco antes de qualquer tentativa de envio — se o envio falhar, ele continua lá, esperando ser tentado de novo.

✓ Sem inconsistência

O aviso só é criado se o dado de negócio também foi salvo — nunca ao contrário.

COMO FUNCIONA, PASSO A PASSO

Fluxograma do fluxo de eventos



Em condições normais, todo o fluxo acontece em segundos — os passos "Rede de segurança" e "Investigação manual" só entram em ação se algo já falhou (ex.: instabilidade momentânea).

RESILIÊNCIA

Quatro camadas de proteção, uma para cada tipo de falha

1. Envio imediato

Caminho normal — leva segundos, cobre a grande maioria dos casos.

2. Verificador automático

Se o envio imediato falhar (ex.: instabilidade momentânea), um processo em segundo plano tenta de novo sozinho, aumentando o tempo de espera a cada tentativa.

3. Disjuntor de segurança

Se uma dependência externa está com problema persistente, o sistema para de tentar por um tempo em vez de insistir — evita piorar uma instabilidade já existente.

4. Fila de investigação

Casos que falharam depois de todas as tentativas ficam separados, visíveis para investigação manual — nada se perde silenciosamente.

DECISÕES DE ARQUITETURA

Escolhas conscientes, não falta de conhecimento

Verificação periódica em vez de leitura contínua do banco

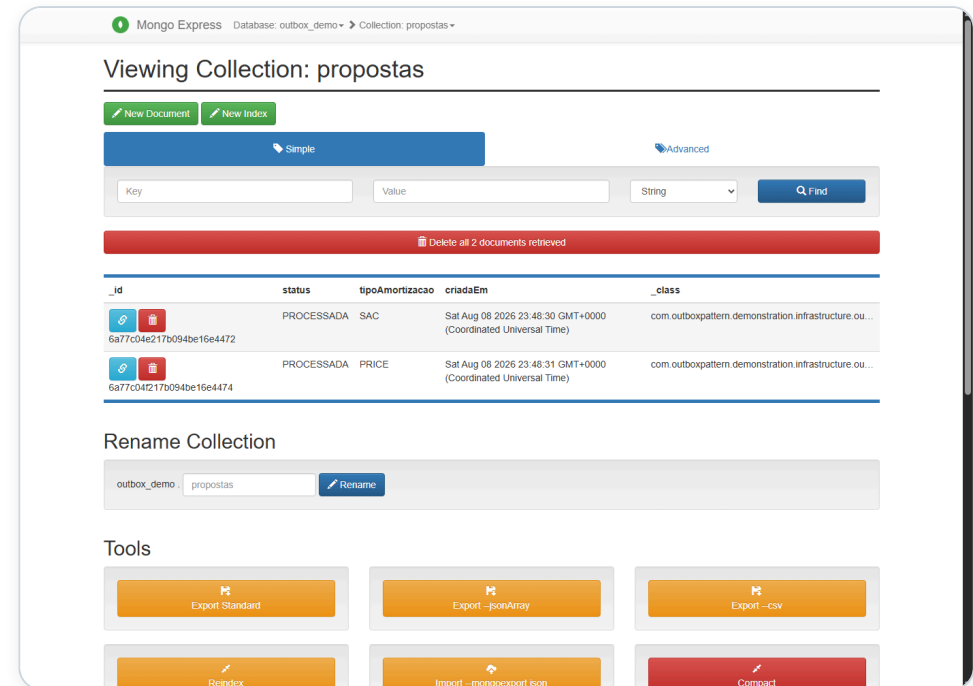
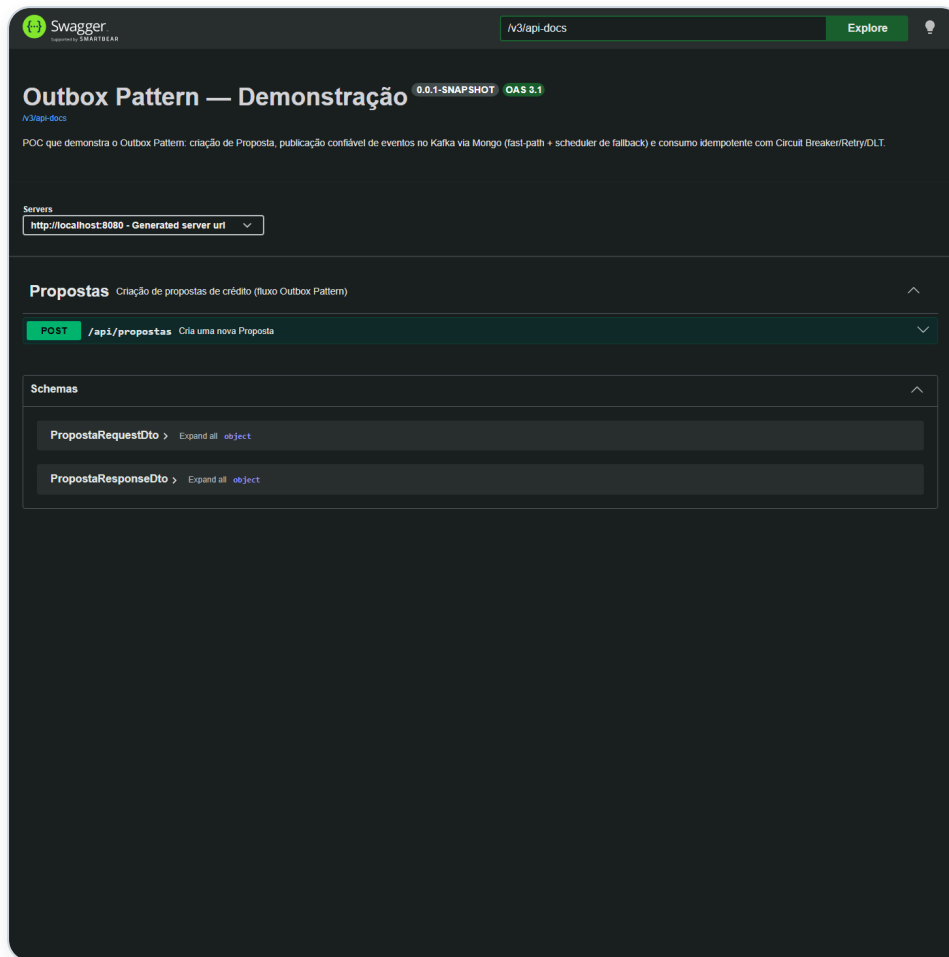
Existe uma técnica mais sofisticada (leitura em tempo real do log do banco) que reduz o tempo de espera de segundos para milissegundos, mas exige mais peças de infraestrutura para operar. Para o objetivo desta demonstração, a verificação periódica é suficiente e mais simples de manter — a técnica mais sofisticada é a recomendação para produção em alta escala.

"Pelo menos uma vez", nunca "exatamente uma vez"

O sistema garante que a mensagem **sempre** chega — o preço disso é que, raramente, ela pode chegar duas vezes. Por isso o lado que recebe já foi construído para reconhecer e ignorar repetições com segurança, sem duplicar nenhum efeito.

PROVAS VISUAIS — FUNCIONANDO DE VERDADE

Criar uma proposta, sem Postman nem terminal



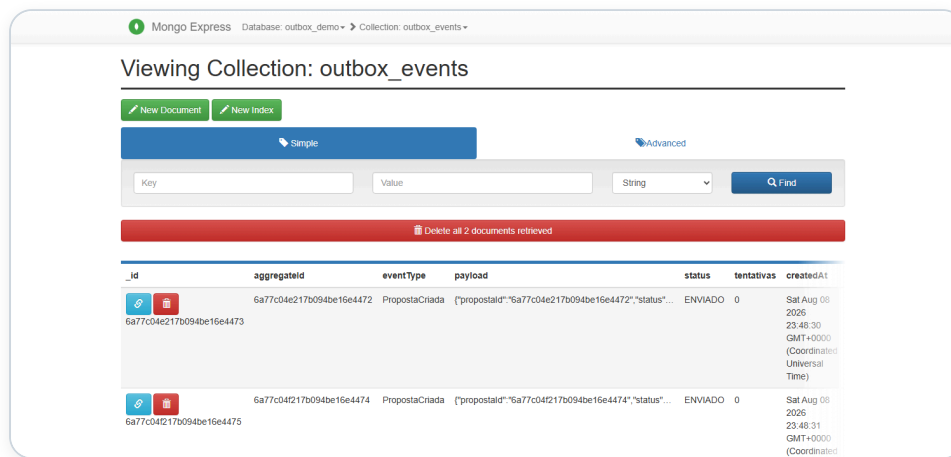
Mongo Express — a proposta salva, já

PROCESSADA

Swagger — qualquer pessoa cria uma proposta clicando em "Try it out"

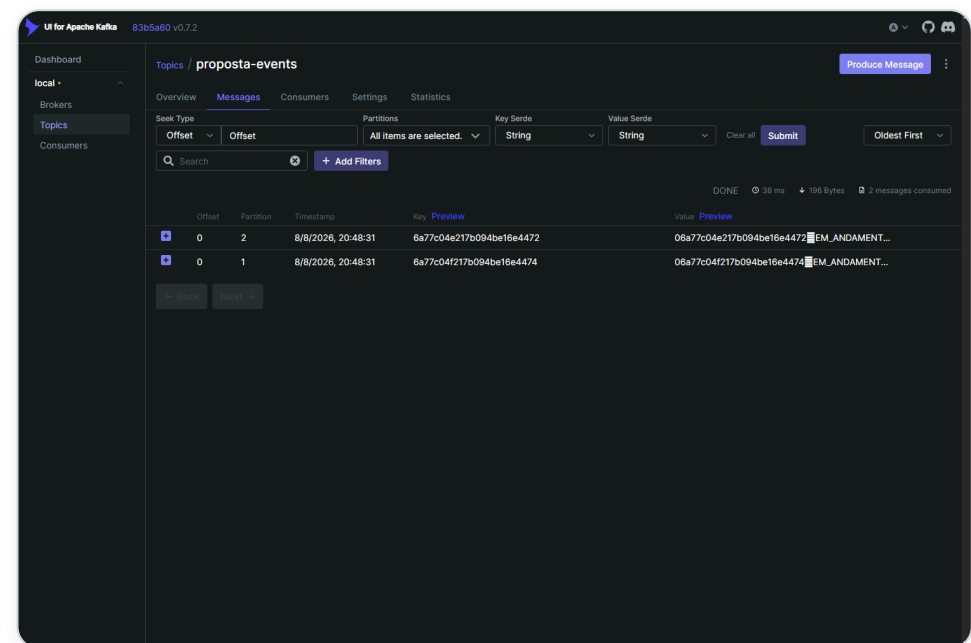
PROVAS VISUAIS — O EVENTO DE VERDADE ACONTECENDO

O aviso registrado e a mensagem entregue



Registro do "aviso pendente" — status

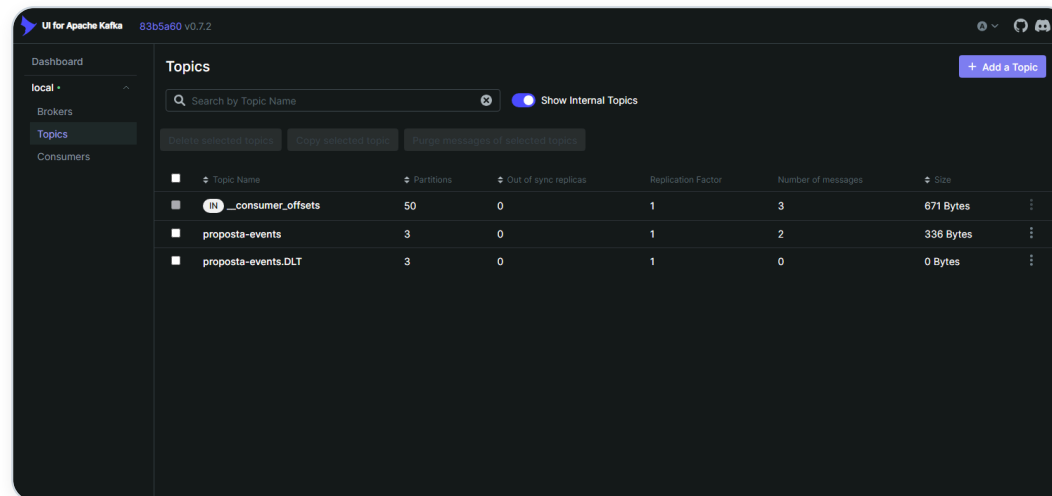
ENVIADO



Kafka UI — as mensagens chegando no canal de eventos

PROVAS VISUAIS — A REDE DE SEGURANÇA EXISTE DE VERDADE

A fila de investigação está lá, pronta, mesmo vazia



UI for Apache Kafka 83b5a60 v0.7.2

Dashboard
local
Brokers
Topics
Consumers

Topics

Search by Topic Name ☒ Show Internal Topics

[Delete selected topics](#) [Copy selected topics](#) [Fetch messages of selected topics](#)

Topic Name	Partitions	Out of sync replicas	Replication Factor	Number of messages	Size
☒ <code>__consumer_offsets</code>	50	0	1	3	671 Bytes
☒ <code>proposta-events</code>	3	0	1	2	336 Bytes
☒ <code>proposta-events.DLT</code>	3	0	1	0	0 Bytes

Canal de eventos e canal de investigação (DLT) provisionados desde o primeiro dia — 0 mensagens na DLT porque nada falhou até aqui

ENTREGA

O que já está pronto

Etapa	Entrega	Status
MVP 0	Orquestração completa — sobe tudo com um comando	✓ Concluído
MVP 1	Criação de Proposta pelo Swagger	✓ Concluído
MVP 2	Outbox Pattern — publicação confiável no Kafka	✓ Concluído
MVP 3	Rede de segurança automática (reprocessamento)	✓ Concluído
MVP 4	Consumo protegido, sem duplicar efeito, com fila de investigação	✓ Concluído
MVP 5	Recursos AWS (Secrets Manager)	Descartado — desnecessário para a demo
MVP 6	Esta documentação	✓ Concluído
MVP 7	Pipeline automatizado com aprovação manual antes de produção	Próximo passo

FECHAMENTO

Sobe com um comando, funciona de ponta a ponta

`docker-compose up -d` — nenhum passo manual, nenhuma configuração extra. O sistema já demonstrou, rodando de verdade: criação de dado + evento na mesma operação, entrega automática, recuperação automática de falha temporária, e uma fila de investigação para o que precisar de atenção humana.

Próximo passo sugerido

Pipeline automatizado (MVP 7) com aprovação manual obrigatória antes de qualquer mudança chegar à branch principal.